DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

User-defined functions and what they cost

Section 1. Python, pandas, and the question of whether you needed one at all.

SECTION 1 of 10 WEIGHT 22% TOPIC UDFs

admin@sthithapragna.com

A user-defined function is a hole in the optimiser. Spark cannot see inside it, so it cannot improve it.

When Databricks says to use one

- **Good fits:** ad hoc queries, manual data cleaning, exploratory analysis, small to medium datasets.
- Common cases named in the docs: **encryption, decryption, hashing, JSON parsing, validation**.
- **Use built-in Spark functions instead** for large datasets, and for anything that runs regularly or continuously.
- That explicitly includes **ETL jobs and streaming**, which is most of what this exam is about.

> ON THE EXAM
>
> Databricks' own guidance is that UDFs suit logic which is difficult to express with built-in functions. Difficult, not merely unfamiliar.

From cheapest to most expensive

01

Built-in

Fully optimised, eligible for vectorised execution. Always the first choice.

›

02

SQL UDF

Expressed in SQL, so the optimiser can still see through it.

›

03

Scala UDF

Runs in the JVM, so no data has to leave it.

›

04

pandas UDF

Leaves the JVM, but in Arrow batches. Up to 100x faster than a Python UDF.

›

05

Python UDF

Row at a time, serialised each way. The most expensive option.

ON THE EXAM

Built-in functions and SQL UDFs are the most efficient. Everything below them pays to move data out of the JVM and back.

Why the plain Python one is so slow

- Every row is **serialised out of the JVM** to a Python interpreter, then the result is serialised back.
- Scalar Python functions rely on **cloudpickle**, which becomes a bottleneck on large inputs and outputs.
- The optimiser cannot reason about it, so filters cannot be pushed through it.
- The vectorised engine cannot accelerate it either, so that part of the plan falls back.

> ON THE EXAM
>
> Four separate costs, all from the same cause: the work is happening outside the engine that was built to optimise it.

The three shapes you should recognise

SHAPE	WHAT IT DOES
Series to Series	Vectorises a scalar operation. Input and output are the same length.
Iterator of Series	Same, but streams batches. Good when set-up per batch is expensive.
Series to Scalar	Collapses many values to one. Used for aggregation with a group by.

ON THE EXAM

The type hint on the function is not decoration. It is how Spark decides which of these three shapes you meant.

The contract of a Series to Series function

- Decorate with **pandas_udf** and declare the return type, for example **double**.
- Annotate the arguments as pandas Series and the return as a pandas Series.
- Returning a **different length** than you received breaks the contract. This shape is row-preserving.
- Inside, you write ordinary vectorised pandas. No loops over rows.

> ON THE EXAM
>
> If the body of a pandas UDF contains a loop over rows, the vectorisation benefit has been thrown away.

Which pandas API fits the shape of your problem

IF YOU NEED	USE
One output row for every input row	A pandas UDF
To process each group as a unit	applyInPandas, on grouped data
A different number of rows out than in	mapInPandas, on the DataFrame
To avoid the pandas conversion entirely	mapInArrow or applyInArrow

ON THE EXAM

Row count is the fastest way to choose. Same number out means a pandas UDF; a different number means one of the map functions.

pandas UDF against grouped apply

PANDAS UDF

Column at a time

- Operates on Series
- One result per input row
- Batches sized by the engine
- Cannot see other rows in a group

APPLYINPANDAS

Group at a time

- Operates on a whole DataFrame per group
- Row count per group may change
- The split, apply, combine pattern
- Loads the entire group into memory

THE TRADE-OFF

The grouped form is powerful because the function sees a whole group. That is also its risk: one huge group can exhaust the executor.

When the map form is the only option

- **mapInPandas** applies a function to an iterator of DataFrames, one batch of a partition at a time.
- Output row count may differ from input. This is the form for exploding, or for heavy filtering.
- It works on the whole DataFrame, so no grouping is involved and no group is held in memory.
- Because it yields batches, it suits work where you cannot hold everything at once.

> ON THE EXAM
>
> If a question describes one input record producing an unknown number of output records, this is the shape being asked for.

The Arrow function APIs

- **mapInArrow** and **applyInArrow** mirror the pandas versions but work on Arrow objects directly.
- They skip the conversion between Arrow and pandas, removing that overhead entirely.
- Arrow-optimised Python UDFs also exist, bypassing the older serialisation path.
- Recognise them. They are not a substitute for using a built-in function in the first place.

> ON THE EXAM
>
> Every improvement in this space narrows the gap with built-in functions. None of them closes it.

Unity Catalog governed against session-scoped

UNITY CATALOG GOVERNED

Shared and permanent

- Persisted in the catalog, reusable across sessions
- Governed by Unity Catalog permissions
- Discoverable in Catalog Explorer
- SQL, Python, Scala and Java

SESSION-SCOPED

Local and temporary

- Exists only in the current Spark session
- Not governed and not shared
- Invisible to anyone else
- SQL, Python and Scala

THE TIEBREAKER

A hashing function used by three teams for compliance should be governed, not redefined in each notebook.

The Unity Catalog types and where they run

TYPE	NOTABLE SUPPORT
Unity Catalog Python UDF	Serverless, standard access mode from DBR 13.3 LTS, SQL warehouses, pipelines.
Batch Unity Catalog Python UDF	Standard access mode from DBR 16.3. Batches values instead of row-by-row.
Unity Catalog Python UDTF	Standard access mode from DBR 17.1. Returns many rows per input row.
Unity Catalog Scala or Java UDF	Also dedicated access mode and classic SQL warehouses.

ON THE EXAM

Nearly all of these require standard access mode on classic compute, with a runtime floor. A dedicated cluster on an old runtime is the usual reason one is unavailable.

Batch Unity Catalog Python UDFs

- Registered in the catalog like any governed function, but it **operates on batches of values**.
- That reduces the row-by-row overhead, which is the whole reason plain Python functions are slow.
- Declared with a pandas parameter style and a named handler, rather than a plain expression body.
- So it gives you governance and vectorisation together, which previously meant choosing one.

> ON THE EXAM
>
> Governed plus vectorised. If a question wants a shared function that also has to run at scale, this is the shape it is describing.

Availability, not syntax

- Non-scalar Python forms need a **higher runtime floor** on classic compute than scalar ones do.
- Scala aggregate functions require **dedicated** access mode, not standard.
- Serverless supports the Python forms broadly, which is one reason it is often the simpler answer.
- A precompiled Java function can be registered from a JAR, subject to the allowlist rules from module 3.

> ON THE EXAM
>
> When a UDF 'does not work' in a question, check the access mode and runtime before questioning the code.

How to test a UDF properly

- Test the **plain Python function** directly, before it is wrapped. It is just a function.
- That runs on a laptop in milliseconds, with no Spark session and no cluster.
- Then test the wrapped version once, on a small DataFrame, to confirm types and null handling.
- Testing only through Spark makes a fast unit test into a slow integration test.

> ON THE EXAM
>
> This is why the module 2 structure matters. A UDF defined inside a notebook cell cannot be imported, so it cannot be tested this way.

What to carry into the exam

- A UDF is **opaque to the optimiser**. That is the reason behind every performance answer here.
- Built-in first, then SQL, then Scala, then pandas, then plain Python.
- Row count picks the API: **same** means pandas UDF, **grouped** means applyInPandas, **different** means mapInPandas.
- Grouped apply loads a **whole group into memory**. Watch for skewed groups.
- Governed for sharing and permissions. Session-scoped for quick local work.

> ON THE EXAM
>
> For a regular ETL or streaming job at scale, the best answer is very often not to use a UDF at all.

A nightly ETL job processes 400 million rows. One step uses a Python UDF that takes a postcode string, strips whitespace, uppercases it, and returns the first half.

The step accounts for most of the job's runtime. The team has been asked to reduce cost without changing the output.

What is the best change?

A Convert it to a pandas UDF so the work is vectorised in Arrow batches.

B Replace it with built-in Spark string functions.

C Register it as a Unity Catalog Python UDF so it can be governed and reused.

D Increase the number of shuffle partitions so the UDF runs on more executors.

◆ PAUSE AND DECIDE

What is the best change?

A Convert it to a pandas UDF so the work is vectorised in Arrow batches.

A real improvement, and the trap. It moves one rung up the ladder when the top rung was available all along.

B **Replace it with built-in Spark string functions.**

Trimming, uppercasing and substring are all built in. Full optimisation, vectorised execution, no data leaves the JVM.

C Register it as a Unity Catalog Python UDF so it can be governed and reused.

Governance is not the problem described, and this does not make it faster.

D Increase shuffle partitions so the UDF runs on more executors.

Buys parallelism with more compute. The per-row serialisation cost is unchanged.

ON THE EXAM

Before optimising a UDF, check whether a built-in function does the job. Trim, upper and substring all exist.

A data scientist needs to fit a small regression per store and return the fitted coefficients, one row per store. There are 1,200 stores. The logic exists in a Python library with no Spark equivalent.

One store is a national distribution centre and holds roughly 60% of all rows. The job fails on that partition with an out-of-memory error.

What best explains the failure?

A applyInPandas loads each entire group into executor memory, and one group is enormous.

B A pandas UDF cannot return fewer rows than it receives, so the job overflowed.

C Arrow serialisation has a fixed batch limit that this group exceeded.

D The Python interpreter cannot hold more than one partition at a time.

◆ PAUSE AND DECIDE

What best explains the failure?

A applyInPandas loads each entire group into executor memory, and one group is enormous.

The defining property of grouped apply, and the reason skewed groups are dangerous with it.

B A pandas UDF cannot return fewer rows than it receives, so the job overflowed.

True of the Series to Series shape, but that is not what a per-group fit uses, and it would not cause an OOM.

C Arrow serialisation has a fixed batch limit that this group exceeded.

Invented. Batch sizing is not the constraint being described.

D The Python interpreter cannot hold more than one partition at a time.

Not a real limitation, and it would not single out one group.

ON THE EXAM

Grouped apply plus a skewed grouping key is a recognisable failure pattern. The dataset size is irrelevant; the largest group is what matters.

> A pipeline reads records where each row contains a compressed archive path. For each row, a Python library must open the archive and emit one output row per file inside it. Some archives contain a single file, others contain thousands.
>
> The team has tried a pandas UDF and cannot make it work.

Which approach fits this problem?

A applyInPandas, grouping by the archive path so each archive is one group.

B A Series to Scalar pandas UDF, aggregating the file list per row.

C mapInPandas, yielding a variable number of rows per input batch.

D A scalar Python UDF returning an array, since arrays can hold many values.

◆ PAUSE AND DECIDE

Which approach fits this problem?

A applyInPandas, grouping by the archive path so each archive is one group.

Contrives a group per row, and still constrains you to what fits in memory per group.

B A Series to Scalar pandas UDF, aggregating the file list per row.

That shape collapses many values to one. This problem needs the opposite.

C mapInPandas, yielding a variable number of rows per input batch.

The row count differs from input to output, which is precisely what this form exists for.

D A scalar Python UDF returning an array, since arrays can hold many values.

Works, then needs an explode, and pays row-at-a-time cost to build a potentially huge array per row.

ON THE EXAM

One input row producing an unknown number of output rows is the signature of the map form.

> Three teams each maintain their own copy of a function that tokenises customer identifiers before analysis. The definitions have drifted apart, and an auditor has asked who is able to run the tokenisation.
>
> The function must also run inside a nightly job over several billion rows, and inside SQL warehouse queries.

Which option best satisfies all of this?

A. A session-scoped pandas UDF, defined in a shared notebook each team imports.

B. A batch Unity Catalog Python UDF, registered once in the catalog.

C. A scalar Unity Catalog Python UDF, registered once in the catalog.

D. A Scala UDAF on a dedicated cluster, called by all three teams.

◆ PAUSE AND DECIDE

Which option best satisfies all of this?

A. A session-scoped pandas UDF, defined in a shared notebook each team imports.

Fast, but ungoverned. Nobody can answer the auditor's question, and drift can recur.

B. A batch Unity Catalog Python UDF, registered once in the catalog.

Governed and permissioned, works on SQL warehouses, and batches values so it scales to billions of rows.

C. A scalar Unity Catalog Python UDF, registered once in the catalog.

Governed and shared, but processes row by row. It solves the audit and not the scale.

D. A Scala UDAF on a dedicated cluster, called by all three teams.

An aggregate function is the wrong shape, and dedicated access mode does not serve three teams well.

ON THE EXAM

Three requirements: one definition, an auditable permission, and scale. The batch governed form is the only option carrying all three.

> A pandas UDF applies a machine learning model to each row. Loading the model from storage takes about four seconds. The current implementation loads the model inside the function body, and the job is far slower than expected.
>
> The output must remain one prediction per input row.

Which change addresses the cause?

A Switch to an Iterator of Series pandas UDF and load the model once per batch.

B Switch to applyInPandas so the model loads once per group.

C Switch to a scalar Python UDF, which caches imports between rows.

D Broadcast the model file to every executor with a cluster library.

◆ PAUSE AND DECIDE

Which change addresses the cause?

A Switch to an Iterator of Series pandas UDF and load the model once per batch.

This shape exists for exactly this: expensive set-up done once per batch, then reused across it.

B Switch to applyInPandas so the model loads once per group.

Invents a grouping the problem does not have, and loads whole groups into memory.

C Switch to a scalar Python UDF, which caches imports between rows.

Moves down the cost ladder, and does not solve the repeated loading.

D Broadcast the model file to every executor with a cluster library.

Makes the file local, but the code still loads it on every call.

ON THE EXAM

Expensive set-up that should happen once per batch rather than once per call is the reason the Iterator shape exists.

> A team writes a Python function that validates an identifier's checksum. They want it unit tested on a laptop in continuous integration, reused by two jobs and one notebook, and running acceptably over 50 million rows nightly.
>
> There is no built-in Spark function for this checksum algorithm.

Which combination best meets the requirements?

A. Define it in a notebook cell and register it with the udf decorator where needed.

B. Put the plain function in the source package, test it directly, and wrap it as a pandas UDF.

C. Write it as a SQL UDF so the optimiser can see through it.

D. Use mapInPandas so the validation can filter out invalid rows as it goes.

◆ PAUSE AND DECIDE

Which combination best meets the requirements?

A. Define it in a notebook cell and register it with the udf decorator where needed.

Not importable, so it cannot be unit tested on a laptop, and it invites the drift of three copies.

B. Put the plain function in the source package, test it directly, and wrap it as a pandas UDF.

The body is testable as ordinary Python, the package is importable by both jobs and the notebook, and the pandas wrapper handles the volume.

C. Write it as a SQL UDF so the optimiser can see through it.

Cheapest if the algorithm can be expressed in SQL. A checksum loop generally cannot.

D. Use mapInPandas so the validation can filter out invalid rows as it goes.

Changes the output by dropping rows. Validation was asked for, not filtering.

ON THE EXAM

This is module 2 and module 4 together: an importable package makes the function testable, and the wrapper makes it fast.